

LAPORAN TUGAS 5

Mata Kuliah Sistem Operasi



Di Susun Oleh :

Muhammad Rafi Amiruddin

Nim (2410651027)

Jurusan Teknik Informatika

Fakultas Teknik

Universitas Muhammadiyah Jember

2025

DAFTAR ISI

LAPORAN TUGAS 5.....	1
DAFTAR ISI	i
1.....	ISI LAPORAN
.....	1
1.1 Tugas 1.....	1
1.1.1 SISTEM BOOKING HOTEL	1

1 ISI LAPORAN

1.1 Tugas 1

1.1.1 SISTEM BOOKING HOTEL

- Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

- Hotel memiliki 5 kamar yang bisa di-booking
- Ada 10 customer yang mencoba booking secara bersamaan (10 threads)
- Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah
- Requirements:
 1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
 2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)
 3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
 4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas).
- Buat file Sistem_Booking_Hotel.c di dalam cd ~/Sistem_booking_Hotel

Yang di dalam nya berisi syntax :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
```

```
#define JUMLAH_KAMAR 5
#define JUMLAH_CUSTOMER 10
#define MAX_COBA 3
```

```

sem_t kamar_tersedia;
pthread_mutex_t mutex_print;
int kamar[JUMLAH_KAMAR] = {0};

int booking_kamar(int id_customer) {
    for (int i = 0; i < JUMLAH_KAMAR; i++) {
        if (kamar[i] == 0) {
            kamar[i] = id_customer;
            return i;
        }
    }
    return -1;
}

void lepas_kamar(int nomor_kamar) {
    kamar[nomor_kamar] = 0;
}

void *customer_thread(void *arg) {
    int id = *(int *)arg;
    int coba = 0;
    int sukses = 0;

    while (coba < MAX_COBA && !sukses) {
        coba++;

        pthread_mutex_lock(&mutex_print);
        printf("👤 Customer %d mencoba booking (percobaan ke-
%d)...\n", id, coba);
        pthread_mutex_unlock(&mutex_print);

        sleep(rand() % 3 + 1);

        if (sem_trywait(&kamar_tersedia) == 0) {
            int nomor_kamar = booking_kamar(id);

            int val;
            sem_getvalue(&kamar_tersedia, &val);

            pthread_mutex_lock(&mutex_print);
            printf("✅ Customer %d BERHASIL booking kamar #%%d\n", id,
nomor_kamar + 1);
            printf("🏠 Kamar tersedia sekarang: %%d\n", val);
            pthread_mutex_unlock(&mutex_print);

            sleep(5);

            lepas_kamar(nomor_kamar);
        }
    }
}

```

```

        sem_post(&kamar_tersedia);

        pthread_mutex_lock(&mutex_print);
        printf("👤 Customer %d CHECK-OUT dari kamar # %d\n", id,
nomor_kamar + 1);
        pthread_mutex_unlock(&mutex_print);

        sukses = 1;
    } else {
        pthread_mutex_lock(&mutex_print);
        printf("❌ Customer %d gagal booking (kamar penuh). Tunggu
2 detik...\n", id);
        pthread_mutex_unlock(&mutex_print);
        sleep(2);
    }
}

if (!sukses) {
    pthread_mutex_lock(&mutex_print);
    printf("😓 Customer %d menyerah setelah %d kali mencoba.\n",
id, MAX_COBA);
    pthread_mutex_unlock(&mutex_print);
}

free(arg);
pthread_exit(NULL);
}

int main() {
    srand(time(NULL));
    pthread_t threads[JUMLAH_CUSTOMER];
    pthread_mutex_init(&mutex_print, NULL);
    sem_init(&kamar_tersedia, 0, JUMLAH_KAMAR);

    pthread_mutex_lock(&mutex_print);
    printf("=== 🏨 SIMULASI SISTEM BOOKING HOTEL ===\n");
    printf("Total kamar: %d | Total customer: %d\n\n",
JUMLAH_KAMAR, JUMLAH_CUSTOMER);
    pthread_mutex_unlock(&mutex_print);

    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&threads[i], NULL, customer_thread, id);
    }

    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

```

pthread_mutex_destroy(&mutex_print);
sem_destroy(&kamar_tersedia);

pthread_mutex_lock(&mutex_print);
printf("\n===  SIMULASI SELESAI ===\n");
pthread_mutex_unlock(&mutex_print);

return 0;
}

```

- Lalu Compile dan jalankan dengan gcc Sistem_Booking_Hotel.c -o Sistem_Booking_Hotel -lpthread dan ./Sistem_Booking_Hotel

Hasil yang di harapkan :

```

=== SISTEM BOOKING HOTEL ===
Total Kamar: 5
[10:30:15]  Customer-1 mencoba booking...
[10:30:15]  Customer-2 mencoba booking...
[10:30:15]  Customer-1 berhasil booking! (4 kamar tersisa)
[10:30:16]  Customer-2 berhasil booking! (3 kamar tersisa)
[10:30:16]  Customer-3 mencoba booking...
[10:30:16]  Customer-10 gagal booking (kamar penuh), mencoba
lagi...
[10:30:20]  Customer-1 check-out (4 kamar tersisa)

```

- Ada 5 kamar hotel dan 10 customer yang mencoba booking.
- Tiap customer dijalankan sebagai thread terpisah, jadi semua bisa booking secara bersamaan (paralel).
- Kalau kamar masih ada, customer berhasil booking dan jumlah kamar berkurang.
- Kalau kamar penuh, customer akan gagal booking, lalu nunggu beberapa detik dan nyoba lagi.
- Setelah beberapa waktu, customer yang sudah dapat kamar akan check-out, bikin kamar kosong lagi.
- Customer lain yang gagal sebelumnya bisa nyoba lagi dan berhasil setelah kamar kosong.
- Di akhir simulasi muncul pesan “SIMULASI SELESAI”, tanda semua thread sudah selesai jalan.

Program ini nunjukkan konsep sinkronisasi thread di C (biasanya pakai mutex atau semaphore), biar gak ada konflik waktu beberapa thread ngakses data bersama (jumlah kamar). Jadi, ini latihan buat memahami sinkronisasi dan manajemen sumber daya bersama dalam pemrograman paralel.

```
rafi@satelit: ~/Sistem_Bookin
rafi@satelit:~$ cd ~/praktikum-sinkronisasi
rafi@satelit:~/praktikum-sinkronisasi$ cd ~/Sistem_Booking_Hotel
rafi@satelit:~/Sistem_Booking_Hotel$ nano Sistem_Booking_Hotel.c
rafi@satelit:~/Sistem_Booking_Hotel$ gcc Sistem_Booking_Hotel.c -o Sistem_Booking_Hotel -lpthread
rafi@satelit:~/Sistem_Booking_Hotel$ ./Sistem_Booking_Hotel
=== 🏠 SIMULASI SISTEM BOOKING HOTEL ===
Total kamar: 5 | Total customer: 10

👤 Customer 1 mencoba booking (percobaan ke-1)...
👤 Customer 2 mencoba booking (percobaan ke-1)...
👤 Customer 3 mencoba booking (percobaan ke-1)...
👤 Customer 5 mencoba booking (percobaan ke-1)...
👤 Customer 6 mencoba booking (percobaan ke-1)...
👤 Customer 4 mencoba booking (percobaan ke-1)...
👤 Customer 8 mencoba booking (percobaan ke-1)...
👤 Customer 10 mencoba booking (percobaan ke-1)...
👤 Customer 7 mencoba booking (percobaan ke-1)...
👤 Customer 9 mencoba booking (percobaan ke-1)...
✅ Customer 6 BERHASIL booking kamar #1
🏠 Kamar tersedia sekarang: 3
✅ Customer 3 BERHASIL booking kamar #2
🏠 Kamar tersedia sekarang: 3
✅ Customer 8 BERHASIL booking kamar #3
🏠 Kamar tersedia sekarang: 2
✅ Customer 7 BERHASIL booking kamar #4
🏠 Kamar tersedia sekarang: 1
✅ Customer 1 BERHASIL booking kamar #5
🏠 Kamar tersedia sekarang: 0

❌ Customer 2 gagal booking (kamar penuh). Tunggu 2 detik...
❌ Customer 5 gagal booking (kamar penuh). Tunggu 2 detik...
❌ Customer 4 gagal booking (kamar penuh). Tunggu 2 detik...
❌ Customer 10 gagal booking (kamar penuh). Tunggu 2 detik...
❌ Customer 9 gagal booking (kamar penuh). Tunggu 2 detik...
👤 Customer 2 mencoba booking (percobaan ke-2)...
👤 Customer 4 mencoba booking (percobaan ke-2)...
👤 Customer 10 mencoba booking (percobaan ke-2)...
👤 Customer 5 mencoba booking (percobaan ke-2)...
👤 Customer 9 mencoba booking (percobaan ke-2)...
👤 Customer 6 CHECK-OUT dari kamar #1
👤 Customer 8 CHECK-OUT dari kamar #3
👤 Customer 7 CHECK-OUT dari kamar #4
👤 Customer 3 CHECK-OUT dari kamar #2
✅ Customer 5 BERHASIL booking kamar #1
🏠 Kamar tersedia sekarang: 3
✅ Customer 9 BERHASIL booking kamar #2
🏠 Kamar tersedia sekarang: 2
👤 Customer 1 CHECK-OUT dari kamar #5
✅ Customer 10 BERHASIL booking kamar #3
🏠 Kamar tersedia sekarang: 2
✅ Customer 2 BERHASIL booking kamar #4
🏠 Kamar tersedia sekarang: 0
✅ Customer 4 BERHASIL booking kamar #5
🏠 Kamar tersedia sekarang: 0
👤 Customer 5 CHECK-OUT dari kamar #1
👤 Customer 9 CHECK-OUT dari kamar #2
👤 Customer 10 CHECK-OUT dari kamar #3
👤 Customer 2 CHECK-OUT dari kamar #4
👤 Customer 4 CHECK-OUT dari kamar #5

=== ✅ SIMULASI SELESAI ===
rafi@satelit:~/Sistem_Booking_Hotel$ |
```

1. Penjelasan bagaimana Anda mengatasi race condition

Jawab :

- Semaphore (room_semaphore) dipakai buat ngatur jumlah kamar yang bisa dipakai bersamaan. Jadi cuma ada 5 “slot” aktif sesuai jumlah kamar. Kalau gak ada semaphore, bisa terjadi overbooking — lebih dari 5 customer bisa booking di waktu bersamaan.
- Lock (room_lock) melindungi proses ambil dan lepas kamar dari daftar kamar tersedia. Tanpa ini, dua thread bisa aja ambil kamar yang sama atau rusak daftar kamar karena akses barengan (race condition).
- Mutex (print_lock) cuma buat merapikan tampilan output, biar teks dari banyak thread gak tumpang tindih di terminal.

2. Apa yang terjadi jika tidak menggunakan sinkronisasi?

Jawab :

- Jumlah kamar bisa melebihi kapasitas (overbooking).
- Dua customer bisa dapet kamar yang sama (data rusak).
- Output terminal acak-acakan.